

CSE 237C Project 3: DFT

Q1: Baseline DFT32

Changing the accuracy of CORDIC core will not change DFT hardware resource use. However, higher accuracy would lead to lower performance of the DFT because of higher latency.

Latency	BRAM	DSP	FF	LUT	Clock Period (ns)	Throughput (MHz)
68068	2	53	5315	9153	7.256	0.00202

Q2: DFT32 with Table Lookup

	Latency	BRAM	DSP	FF	LUT	Clock Period (ns)	Throughput (MHz)
Baseline	68068	2	53	5315	9153	7.256	0.00202
Using LUT	257	66	640	59208	91730	7.256	0.53625

As we can see, with the use of LUT, the latency and throughput significantly improved but the resource usage also increased considerably.

Q3: DFT32 with Interface Change

- We separated the input arrays from the output arrays to avoid overwriting input values during computation. With the original design, there was limited parallelism. By having separate memory interfaces for the output arrays, the DFT reads from one interface and writes to another. As a result, the HLS tool can apply more optimizations such as pipelining and loop unrolling, and array partitioning. This enables multiple input values to be read in parallel while only a single output is written each iteration, reducing latency.

b.

	Latency	BRAM	DSP	FF	LUT	Clock Period (ns)	Throughput (MHz)
Combined Arrays	257	66	640	59208	91730	7.256	0.53625
Separated Arrays	216	64	620	58034	89476	7.256	0.63804

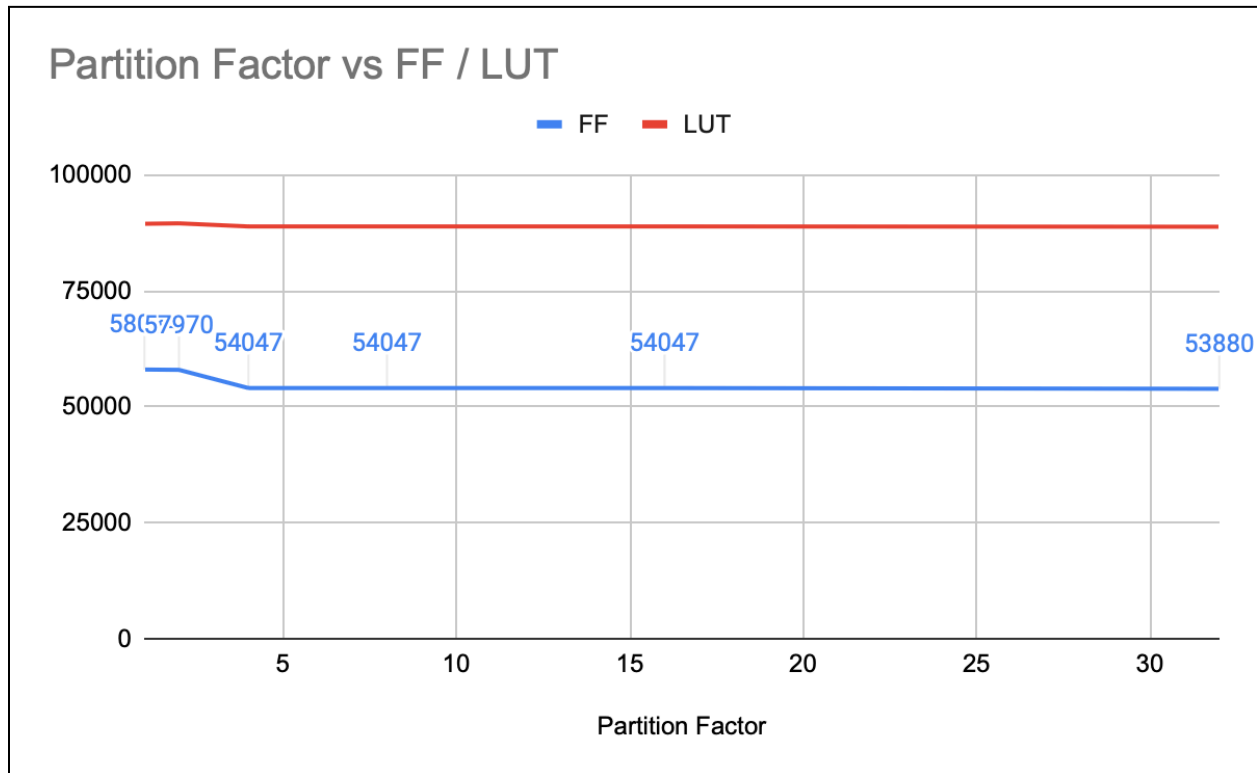
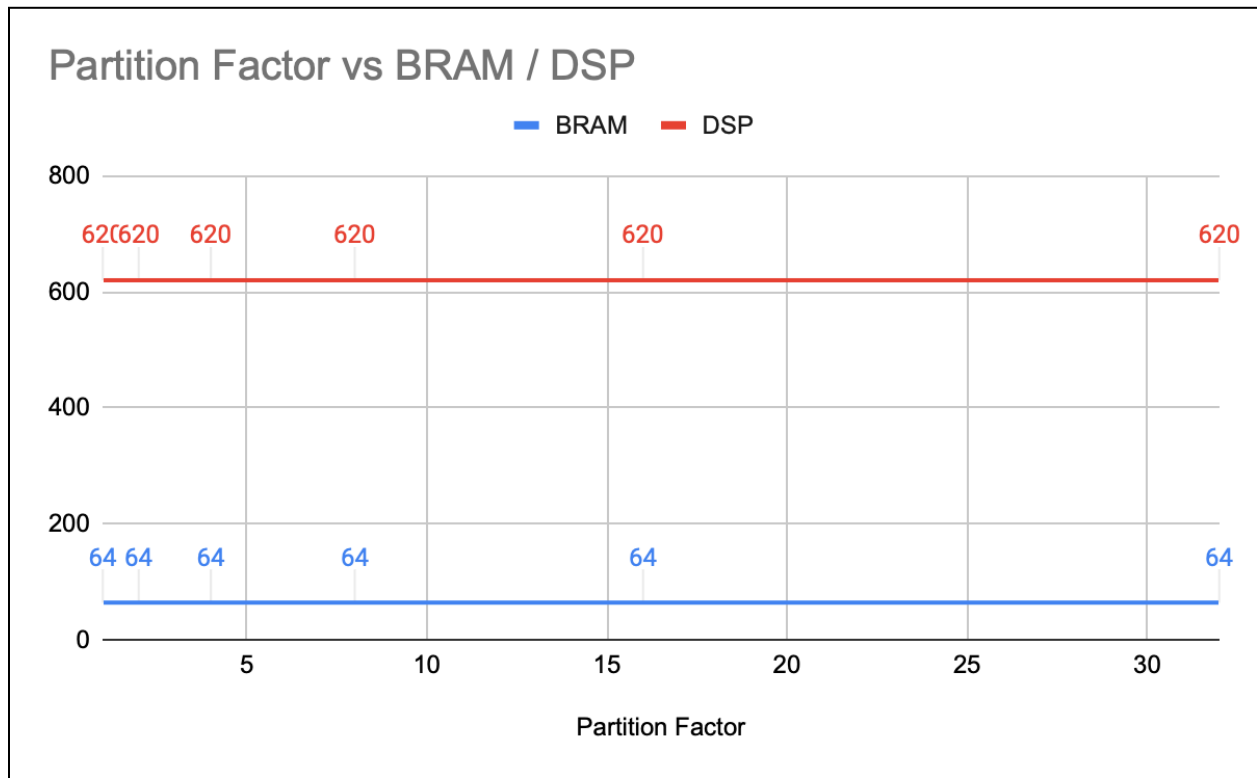
- c. Results: Separating the input and output arrays reduce the latency from 257 to 216 cycles. It also improved the throughput. Hence, there was an increase in performance. The resource usage reduced very slightly, it remained almost the same. Hence we do not observe a significant loss in area.

Q4: DFT32 with Array Partitioning

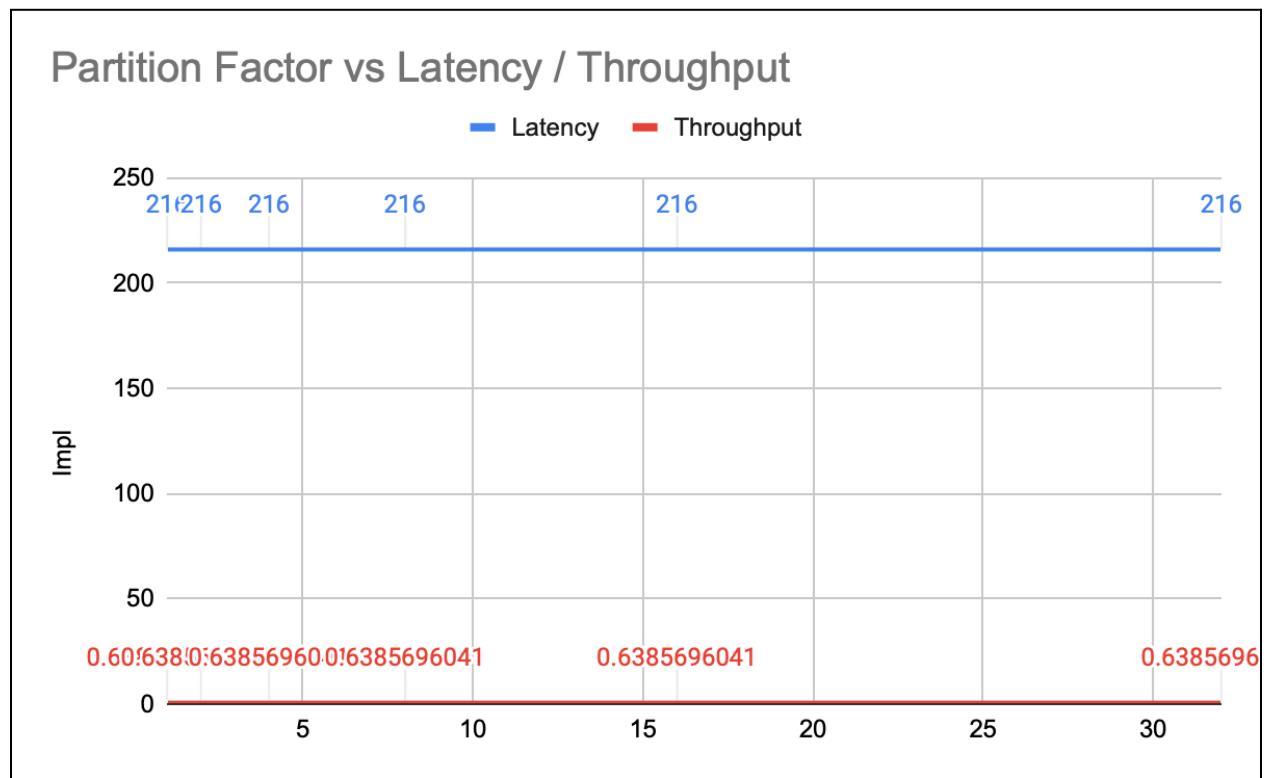
a.

Partition Factor	Latency	Interval	BRAM	DSP	FF	LUT
1	216	217	64	620	58034	89467
2	216	217	64	620	57970	89557
4	216	217	64	620	54047	88890
8	216	217	64	620	54047	88890
16	216	217	64	620	54047	88890
32	216	217	64	620	53880	88826

b.



c.



Q5: DFT32 with Array Partitioning and Loop Unrolling

a.

Here, we used array partitioning factor **32** as it provided the best latency in Q4.

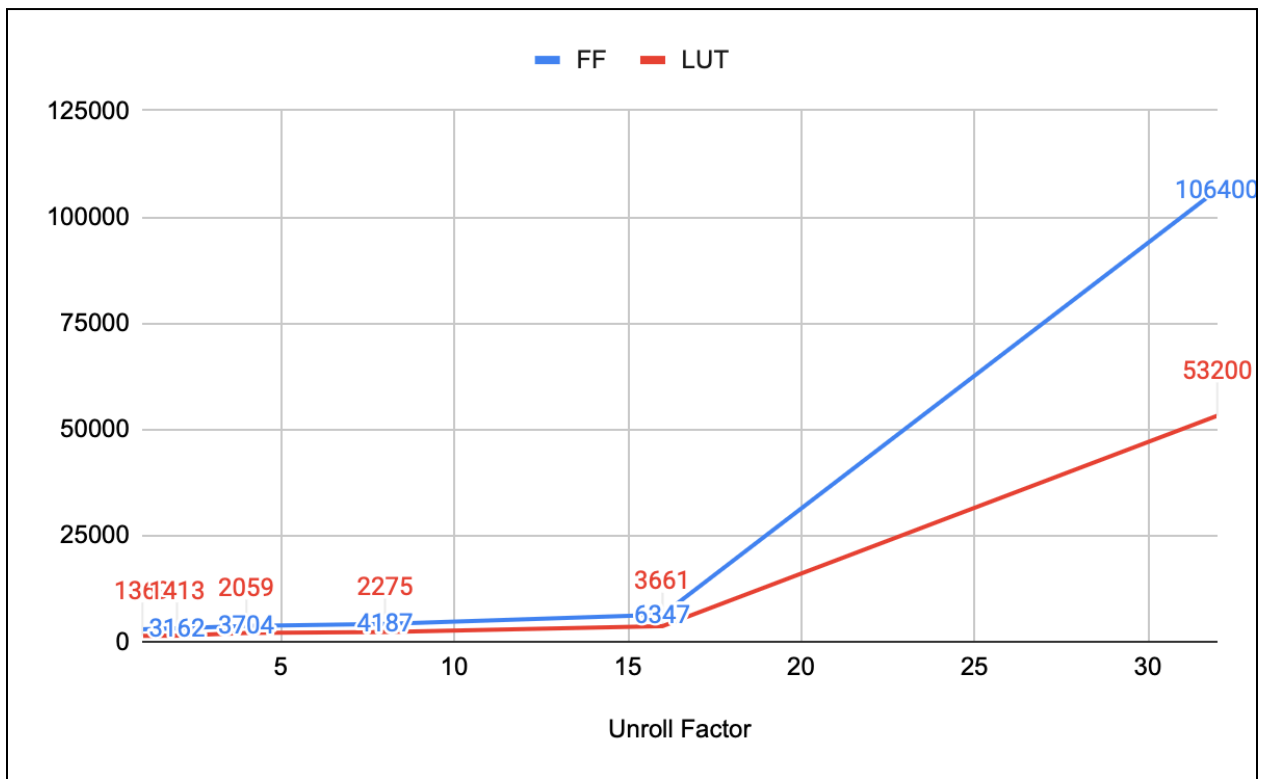
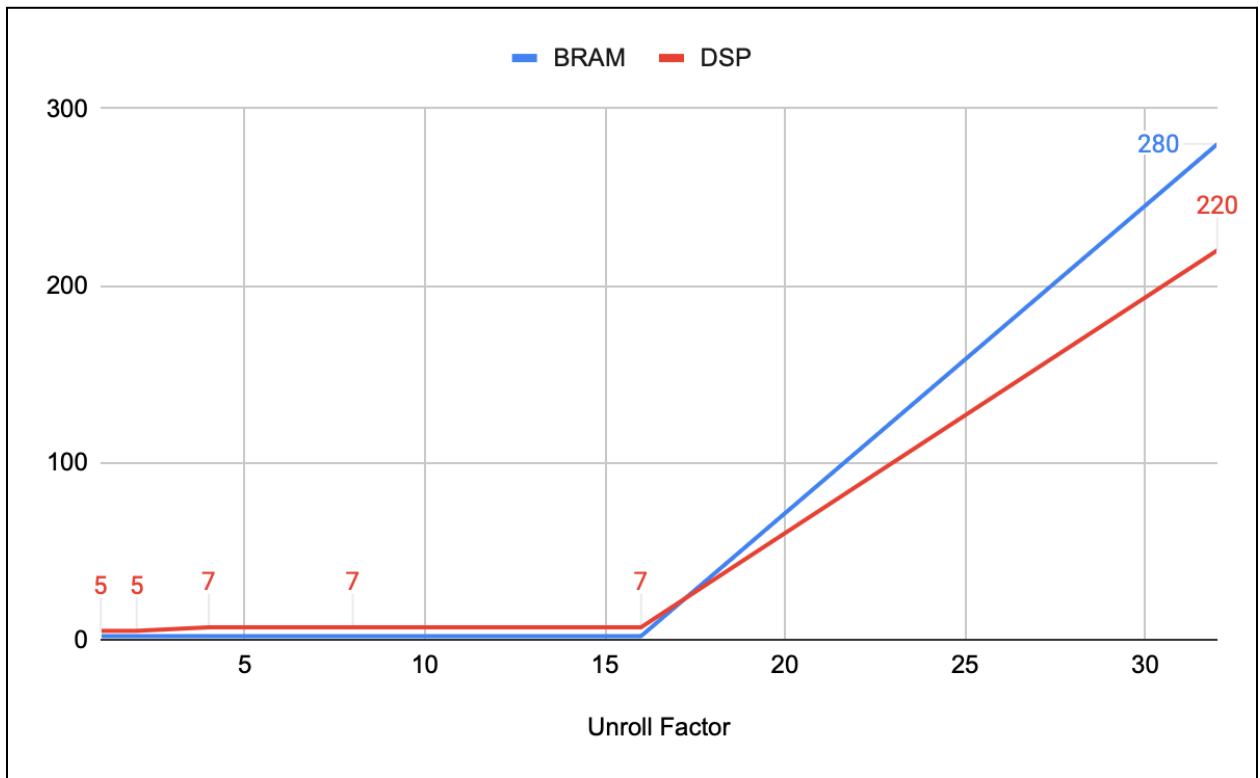
We confirmed the HLS compiler was successfully unrolling the loops from factor=2:

```
INFO: [HLS 214-188] Unrolling loop 'VITIS_LOOP_17_2' (dft.cpp:17:26) in function 'dft' partially with a factor of 2 (dft.cpp:6:0)
```

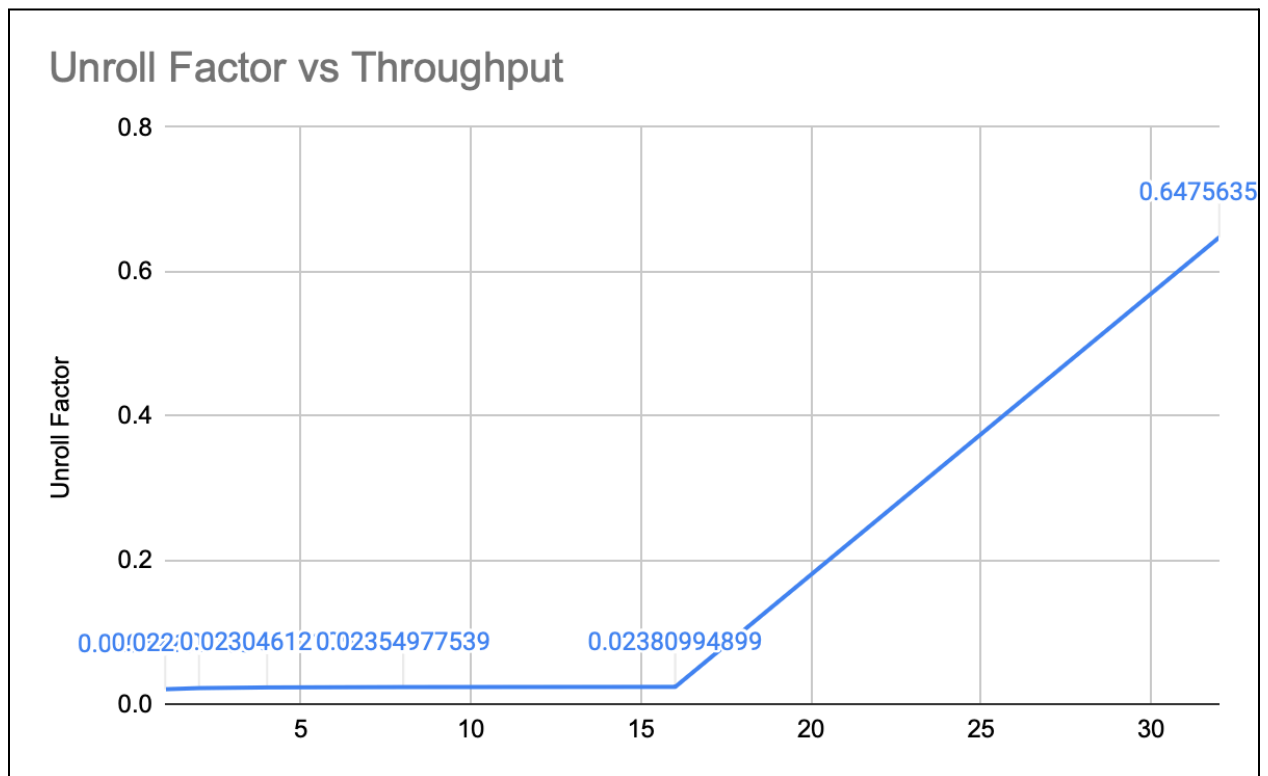
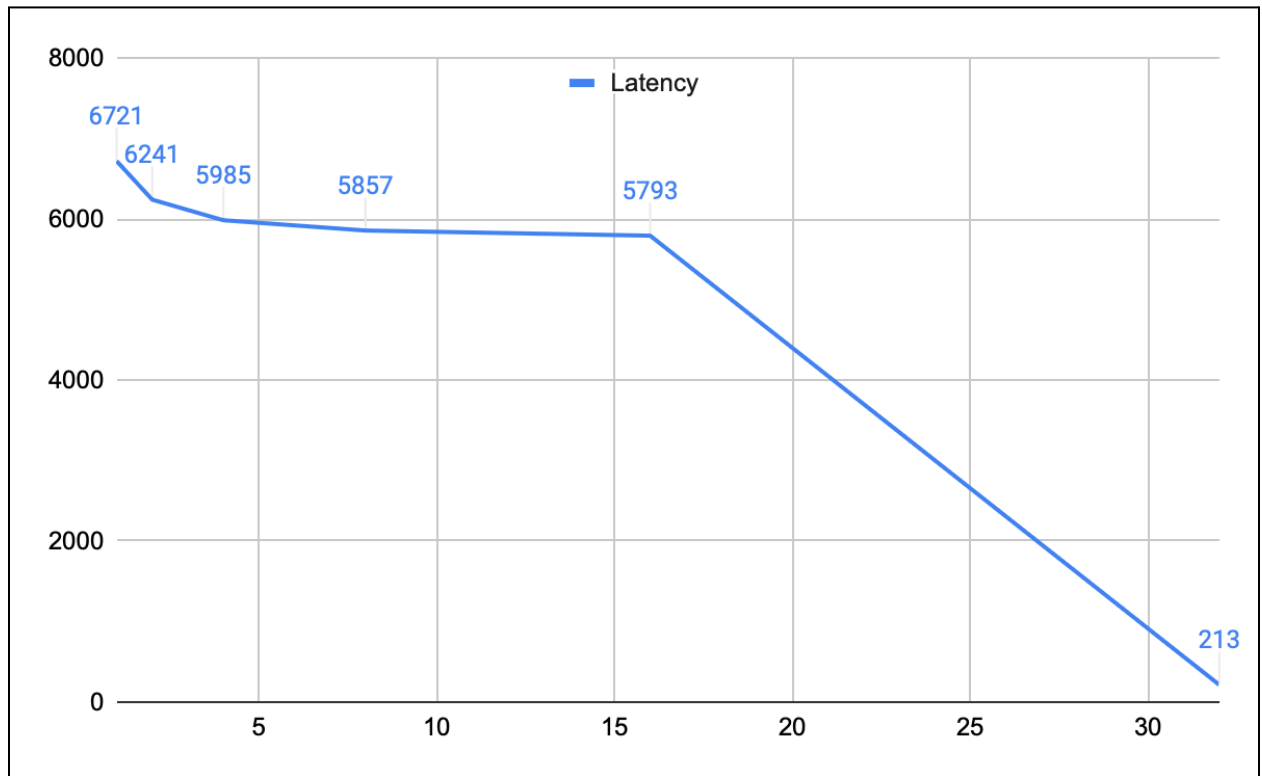
Impl	Latency	Interval	BRAM	DSP	FF	LUT
No unroll	6721	6722	2	5	2923	1369
2	6241	6242	2	5	3162	1413
4	5985	5986	2	7	3704	2059
8	5857	5858	2	7	4187	2275
16	5793	5794	2	7	6347	3661
32	213	214	280	220	106400	53200

With a loop unrolling factor of 32, the **entire loop** could be unrolled and executed in parallel. This explains the sudden, huge rise in resource usage (and 25x decrease in latency).

b.



C. Unroll factor vs Latency



Q6: DFT1024 Baseline

a. Implementation using sin() and cos()

Latency	Throughput	BRAM	DSP	FF	LUT
102784001	0.000001341 950431	16	220	14892	17828

b. Implementation using cos/sin coefficients table

Latency	Throughput	BRAM	DSP	FF	LUT
6291985	0.000021921 70428	4	5	1231	1422

Q7: DFT1024 Loop Optimization

Latency	Throughput	BRAM	DSP	FF	LUT
1049107	0.0001314747061	4	20	2692	16775

Q8: DFT1024 Best Design

- a. For the best design, we've interchanged the inner and outer loops to remove the RAW dependency present in the original design
We also unrolled the inner loop by 8 as that provided the best trade-off between the latency and resource usage. To accommodate the unrolling, we partitioned the imag_op and real_op arrays by factor 8.

b.

Latency	Throughput	BRAM	DSP	FF	LUT
131092	0.001052169732	16	160	17434	24387

Q9: Streaming Interface Synthesis

Latency	Throughput	BRAM	DSP	FF	LUT
6292502	0.00002191990316	8	5	1133	1538